

Module 4 — CRUD with the MVC Pattern

Time: 1.5 hours · Covers Practical 2 · CO2, CO4

Practical 2: Develop a simple CRUD (Create, Read, Update, Delete) application using Node.js and a MongoDB database.

Part A — What is CRUD? (concept, ~10 min)

CRUD is the four basic operations you can do on data. Each maps to an HTTP method and a Mongoose call:

Operation	HTTP method	Route	Mongoose call
Create	POST	/api/tasks	Task.create(...)
Read	GET	/api/tasks	Task.find()
Read one	GET	/api/tasks/:id	Task.findById(id)
Update	PUT	/api/tasks/:id	Task.findByIdAndUpdate
Delete	DELETE	/api/tasks/:id	Task.findByIdAndDelete

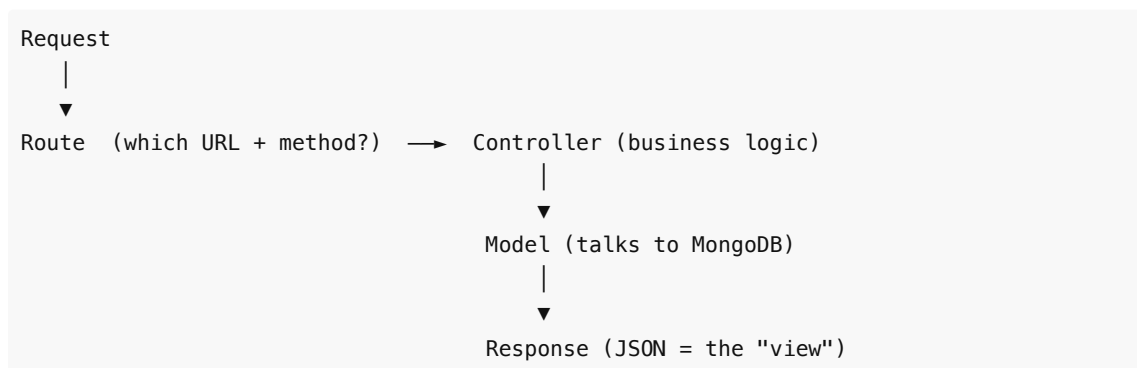
We will build a **Task** API supporting all five.

Part B — What is MVC? (concept, ~15 min)

MVC = Model–View–Controller. It is a way to *organize* code by responsibility, so a project stays understandable as it grows.

- **Model** — the data and its rules (the Mongoose schema/model). *Knows nothing about HTTP*.
- **View** — how data is presented. For a JSON API, the "view" is simply the JSON we send back. (In a server-rendered app it would be HTML templates.)
- **Controller** — the logic that ties them together: it receives the request, calls the model, and sends the response.

Add **Routes** as a thin layer that maps URLs to controller functions.



Why bother? Without MVC, everything lands in one giant file: routes, validation, DB queries, and formatting all tangled together. With MVC, each file has ONE job, so code is easier to find, test, and change. (*This is CO2.*)

Part C — The folder structure (~10 min)

```
04-crud-mvc/
├── src/
│   ├── models/
│   │   └── task.model.js      # M: the Task schema + model
│   ├── controllers/
│   │   └── task.controller.js # C: create/read/update/delete logic
│   ├── routes/
│   │   └── task.routes.js    # maps URLs -> controller functions
│   ├── db.js                # connects to MongoDB
│   ├── app.js               # builds the Express app + wires routes
│   └── server.js            # starts the server
```

Trace one request through the layers:

1. `server.js` starts the app from `app.js` .
2. `app.js` says "any `/api/tasks` URL → use `task.routes.js` ".
3. `task.routes.js` says " `POST /` → `taskController.create` ".
4. `task.controller.js` runs the logic and calls the **Model**.
5. `task.model.js` (the Model) reads/writes MongoDB.
6. The controller sends JSON back (the "View").

Read the files in this order — they are all commented: [task.model.js](#) → [task.controller.js](#) → [task.routes.js](#) → [app.js](#) → [server.js](#) .

Part D — Run and test the CRUD API (~40 min)

```
cd 04-crud-mvc
npm install
cp .env.example .env      # edit with your MongoDB connection string
npm start                 # starts on http://localhost:4000
```

Test every operation with `curl` :

```
# CREATE
curl -s -X POST http://localhost:4000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Learn MVC","priority":"high"}'

# READ all
curl -s http://localhost:4000/api/tasks

# READ one (replace <id> with an _id from the create/read output)
curl -s http://localhost:4000/api/tasks/<id>

# UPDATE (mark it done)
curl -s -X PUT http://localhost:4000/api/tasks/<id> \
  -H "Content-Type: application/json" \
```

```
-d '{"completed":true}'  
  
# DELETE  
curl -s -X DELETE http://localhost:4000/api/tasks/<id>
```

Notice how each controller returns the **right status code** (from Module 2): **201** on create, **200** on read/update, **404** when the id does not exist, **400** on invalid input.

Summary

- **CRUD** = Create, Read, Update, Delete — the four data operations.
- **MVC** separates code by job: **Model** (data), **View** (JSON output), **Controller** (logic), with **Routes** mapping URLs to controllers.
- Each layer has one responsibility, so the app scales cleanly.
- Correct HTTP status codes make the API predictable.

Now do [practice.md](#) .